

REPORT FINAL ASSESSMENT: HOURLY RIVER STREAMFLOW FORECASTING (TRACK 1)

GROUP DLX

**MCTA 4363
DEEP LEARNING**

INSTRUCTOR

ASSOC. PROF. TS. DR. HASAN FIRDAUS BIN MOHD ZAKI

SEMESTER 2 25/26

TEAM MEMBERS :

NAME	MATRIC NO.
MUHD AKMAL HAKIM BIN SAIFUDDIN	2216093
AHMAD HAZAMI BIN MOHD RAZIP	2211203
MUHAMMAD ARIEF ZULKARNAIN BIN ABDUL RAHMAN	2217275

SUBMISSION DATE: 19TH JUNE 2026

TABLE OF CONTENT

INTRODUCTION.....	3
METHODOLOGY.....	3
Datasets.....	3
System Architecture.....	3
RESULTS.....	3
DISCUSSION.....	3
Limitation and Challenges.....	3
Lesson learned.....	3
CONCLUSION.....	3

INTRODUCTION

Forecasting of river streamflow is an important hydrological activity for irrigation management, reservoir operation, flood prediction, and water resource planning. Well-made forecasts can help authorities to mitigate the effects of floods on infrastructure and people and prepare for catastrophic weather events. However, predicting river discharge is difficult due to complex and nonlinear interactions between temperature, rainfall, basin characteristics, and past streamflow.

The objective of this project was to design a deep learning model that can accurately predict hourly river streamflow for the Kaggle Hourly River Streamflow Forecasting Hackathon. Competition participants were asked to forecast streamflow for four forecasting horizons: the present hour ($Q(t)$), one hour ahead ($Q(t+1)$), two hours ahead ($Q(t+2)$), and three hours ahead ($Q(t+3)$). The model performance was evaluated by Root Mean Squared Error (RMSE), where a lower RMSE indicates better prediction accuracy.

In this study, three different deep learning architectures were designed and evaluated. With each trial, the model gradually surpassed the previous one, using longer sequences of historical data, more complex neural network topologies, and better feature engineering. The team's ranking on the Kaggle leaderboard improved from 54th place in the first experiment to 42nd place in the final submission with the final model.

METHODOLOGY

Datasets

The dataset was obtained from the Hourly River Streamflow Forecasting Hackathon. Four CSV files were provided for training and testing.

TrainTrackA.csv

- Contains data from five river basins.
- Covers January 2005 until December 2015.
- Includes the target discharge variable (qobs_mm_per_hour).

TestTrackA.csv

- Covers January 2017 until December 2018.
- Target discharge values were removed.
- Participants were required to generate predictions for the year 2018.

TrainTrackB.csv

- Contains historical observations from one river basin.
- Covers January 2015 until December 2015.

TestTrackB.csv

- Covers January 2017 until December 2018.
- Used for prediction only since the target values were unavailable.

The datasets also contained multiple hydrological and meteorological attributes describing rainfall, temperature, and basin characteristics, which were used as input features for model training.

System Architecture 1 (Experiment 1)

The first experiment served as the baseline model to verify that Long Short-Term Memory (LSTM) networks could effectively model river streamflow as a time-series forecasting problem.

Data Pre-processing

- Sequence length: 24 hours
- Feature scaling using StandardScaler
- Target transformed using $\log_{1p}(q_{obs})$

Feature Engineering

Only the original numerical variables provided in the dataset were used without additional feature extraction.

Model Architecture

LSTM (64) → Dropout (0.2) → Dense (32) → Dense (4)

Training Strategy

The baseline model was trained with the Mean Squared Error (MSE) loss function, which is widely used in regression problems and penalizes larger prediction errors more heavily. Adam optimizer was chosen because of its effective convergence for deep learning models and flexible learning rate.

To assess model performance during training, Root Mean Squared Error (RMSE) was tracked on the validation data set. The Kaggle competition also used RMSE as the evaluation metric, so by monitoring this metric we ensured that the goal of training was aligned with the competition requirements.

To prevent overfitting, the model also used EarlyStopping which automatically stopped training whenever the validation RMSE did not improve for a few consecutive epochs. This helped to save the model with the best validation accuracy and avoid unnecessary training.

The output values were then converted back to the original discharge scale by applying the inverse of the $\log_{1p}$ transformation. This post-processing step ensured the projected streamflow values were directly comparable to the actual observations.

This baseline model achieved a Kaggle leaderboard ranking of **54th place**.

System Architecture 2 (Experiment 2)

The second experiment significantly enhanced the forecasting capability by incorporating richer feature engineering and a deeper recurrent neural network architecture.

Improvements

- Sequence length increased from 24 hours to 72 hours.
- MinMaxScaler used for both input features and target values.
- Added lag features.
- Added rolling mean and rolling standard deviation.
- Added time-related features.

Model Architecture

Bidirectional LSTM (128) → Dropout LSTM (64) → Dropout Dense (64) → Dense (32) → Dense (4)

Training Strategy

In order to enhance model convergence and generalization, the second experiment used a more thorough training approach. The model was optimized using the Adam optimizer with Mean Squared Error (MSE) as the loss function, same like in the first experiment.

Throughout training, a number of evaluation criteria were tracked, such as:

- **Mean Absolute Error (MAE)** to measure the average prediction error.
- **Root Mean Squared Error (RMSE)** to evaluate prediction accuracy while emphasizing larger errors.
- **Coefficient of Determination (R^2)** to assess how well the model explained the variance in streamflow observations.

Several callback functions were introduced to improve training efficiency:

- **EarlyStopping** monitored the validation loss and stopped training when no further improvement was observed, reducing the risk of overfitting.
- **ReduceLROnPlateau** automatically reduced the learning rate whenever the validation performance stagnated. Lowering the learning rate allowed the optimizer to perform finer parameter updates and escape local minima.
- **ModelCheckpoint** continuously saved the model with the best validation performance, ensuring that the final submission used the most accurate model obtained during training.

The extra designed features provided more contextual information to learn temporal connections. The longer input sequence (72 hours) allowed the network to capture delayed hydrological responses.

Post-processing

- Negative prediction clipping
- Median filling
- Prediction smoothing

Negative streamflow predictions were not possible, and therefore negative discharge predictions were adjusted to zero. Smoothing the predictions reduced sudden changes between two subsequent prediction periods, while median values of predictions were used for replacing unstable predictions.

These improvements increased the leaderboard ranking to **43rd place**.

System Architecture 3 (Experiment 3)

The final experiment introduced a hybrid deep learning architecture combining convolutional and recurrent neural networks to capture both local and long-term temporal patterns.

Additional Feature Engineering

- Rainfall rolling mean
- Rainfall rolling sum
- Temperature rolling mean

Model Architecture

Conv1D → Bidirectional LSTM → GRU → Dense (4)

With noise reduced, the Conv1D layer helps to capture local temporal patterns. While the GRU layer increases memory learning through computing efficiency, the Bidirectional LSTM is able to capture the long-term dependencies of sequences in two directions.

Training Strategy

This optimal approach was then used for the more complex hybrid network architecture in the final experiment. RMSE, MAE, and R2 were used to measure validation performance, while the model was optimized through the Adam method using MSE as the loss function.

A number of methods of regularizing and optimizing the model were used to ensure its robustness and prevent overfitting:

- **Dropout layers** were inserted between neural network layers to randomly deactivate neurons during training, encouraging the model to learn more generalized feature representations.
- **EarlyStopping** terminated training once validation performance stopped improving, preventing unnecessary epochs and reducing overfitting.
- **ReduceLROnPlateau** dynamically lowered the learning rate whenever validation performance plateaued, allowing the optimizer to make smaller and more precise parameter updates.

- **ModelCheckpoint** saved the best-performing model throughout training so that the final predictions were generated using the optimal network weights.

Due to the higher model complexity, care had to be taken during the process of integration of the two processes due to the need for optimization. The selected method of training ensured good generalization on unseen data while providing convergence.

Post-processing

- Clipping negative predictions
- Median filling
- Prediction smoothing
- Submission file generation

As per the forecast, smoothing methods were applied to reduce any artificial variance in future forecasts, negative discharge values were set to zero, and unstable forecasts were corrected through median imputation. The last stage involved formatting the processed forecasts into the required Kaggle submission format.

This hybrid architecture achieved the team's highest Kaggle ranking of 42nd place.

RESULTS

The project showed continuous improvement throughout the three experiments.

Experiment	Model	Kaggle Ranking
1	LSTM	54th
2	Bidirectional LSTM	43rd
3	Conv1D + BiLSTM + GRU	42nd

The progression in ranking demonstrates that every architectural improvement contributed positively to forecasting performance.

Major improvements included:

- Longer historical sequence windows.
- More informative engineered features.
- Better feature scaling.
- Hybrid deep learning architecture.
- Improved optimization techniques.
- Better prediction post-processing.

DISCUSSION

River streamflow can be modeled as a time-series forecasting problem, as the baseline LSTM effectively showed. However, prediction accuracy was hindered by its short sequence length and absence of feature engineering.

In the second experiment, performance was significantly enhanced by adding lag features and rolling statistics, which allowed the model to access historical hydrological behavior rather than just immediate data. By taking into account data from both temporal directions, the Bidirectional LSTM improved sequence learning even further.

The best model was the Conv1D-BiLSTM-GRU hybrid architecture. Although the GRU helped increase memory efficiency through fewer parameters than deep LSTM, Conv1D was able to extract local temporal information and pass them on to the recurrent layers. Generalization and decreased prediction errors were some of the benefits of this combination.

It is clear from the gradual improvement in the rank from 54 to 42 that the performance of deep learning models for hydrological forecasting can be improved significantly through effective feature engineering and architecture improvement.

Limitation and Challenges

Challenges encountered during this study included:

a) Very High Peak Flow Events

Even though they occur rarely, floods have a huge effect on the root mean squared error. The penalty incurred for small forecast mistakes when such floods occur is exceedingly large.

b) Hydrological Interdependencies Over Time

Basin response delay and prior rainfall events have a great influence on the flow of rivers. Recurrent neural networks and increased input lengths were therefore needed in order to capture the interdependencies between different time points.

c) Computing Power Requirements

Training of large recurrent neural networks required enormous memory and computing capacity. There was also increased risk of overfitting due to increase in model complexity, hence the need for regularization methods such as early stopping and dropout.

Lesson learned

The development of deep learning models capable of addressing realistic problems associated with time series forecasting became possible due to this work. We learned that, in terms of performance prediction, both feature engineering and preprocessing of data are as important as model design itself.

Besides, the experiment demonstrated the importance of careful refinement of a model rather than relying solely on deep neural networks. The leaderboard performance significantly increased due to thorough research involving sequence length, feature engineering, hybrid architectures, and optimization.

Finally, participation in a competitive Kaggle environment provided us with a better understanding of machine learning practice in the real world.

CONCLUSION

In this study, three deep learning models for hourly prediction of river streamflow were successfully created and evaluated. In the beginning of the project, the initial version of the baseline based on LSTM and finally the hybrid Conv1D-BiLSTM-GRU model capable of capturing both long-range hydrological dependences and local temporal dependencies was proposed.

As a result of continuous development and improvements in the field of preprocessing, feature extraction, neural network architecture, and training methodology, the position on the Kaggle leaderboard was constantly increasing from 54th to 43rd and finally 42nd place. These results prove that the accuracy of predictions of complex time series data can be significantly enhanced through the usage of hybrid deep learning approaches along with proper feature engineering.

Therefore, not only was the project objective of constructing an effective streamflow forecasting model reached but also participants were provided with practical skills in the field of deep learning model creation and machine learning application.